

Reinforcement Learning Applied Project : Healthy Walk

Mikael TOM (0654593)

June 4, 2026

1 Introduction

Walking has interesting benefits on mood and health. Some people may think that any walk is a good walk since the action of walking is just a psychomotor activity described as repeating steps. However, in reality, the feeling afterward may completely change depending on the chosen path. Yet, finding a good path for a healthy walk can be difficult. People usually follow the same path with the same goals such as going to school, to the library or to the grocery store. In addition, there are some places where the infrastructure makes going for a walk even harder. For instance, the coastal city of Ulqin in Montenegro where sidewalks are rare and there is no clear organization for pedestrians. There is a low walking culture there, people prefer taking their car for a 500m ride rather than walking.

Given all that, the objective of this project is to design an OpenAI Gym environment that proposes, once the PPO algorithm Stable-baselines3 agent learned, to a user from Ulqin interesting walks starting and ending at their current location. The duration of the walk can be between 5 and 50 minutes, depending on the user's needs.

2 Methodology

2.1 What is a "Good Walk" ?

Before designing the environment, it was necessary to define clearly what a "good walk" is. Saelens and Handy, 2008 pointed out that accessibility of amenities (shops or services), a pleasant environment and pedestrian safety are essential criteria for a walkable environment. Based on this intuition, we used OSMnx and OpenStreetMap tags to retrieve these interesting locations in Ulqin. The nodes representing amenity are found using the "amenity" tag. These are places such as shops, restaurants or services.

We used tags such as "nature", "parks", "water", "cliff", "dunes", "beach" or "valley" to capture the nodes that represent a pleasant natural environment. Additionally, we added nodes related to places that must absolutely be visited, for instance, monuments and historical places. The idea was to capture a pleasant environment different than just nature.

Furthermore, nodes linked to pedestrian safety are managed with two strategies. The first group contains nodes that the agent must avoid. They are associated with unpleasant places for pedestrians. These include high-traffic highways, high-speed roads ($\geq 50 \text{ km/h}$), difficult paths with bad smoothness, tracks of poor quality (mud or dirt) and land uses perceived as a threat (cemetery or military lands). The second groups contain nodes describing safe places for pedestrians such as pedestrian zones or living streets with sidewalks or crosswalks, and a speed limit of a maximum of 40 km/h .

These different nodes represented in *GeoDataFrames* are kept in a *json* file that is loaded by the designed environment.

2.2 What did not work ?

During the development of this project, multiple strategies were tested. At the beginning, the agent had to pick any node and it was linked via the shortest path to the last node selected. There were major problems with that. First, the agent picked nodes way too far, making the episode end with only one action. With that, the agent was not able to learn because the episodes were too short. Having more than 20-30 actions is considered challenging in Reinforcement Learning. As a result, the agent struggled to learn an optimal policy.

After that, the intuition was to let the agent choose a distance between $100m$ and $1000m$ with an angle between 0 and 359 degree. Specifically, using the last selected node in the current path as the source, the closest node to the chosen distance and angle was selected. Then, it was linked via the shortest path to the source. However, this approach failed because it introduces variability in the approximations depending on the position on the map. In dense areas in the graph, a small variation of angle or distance can drastically shift the selected node. In contrast, larger variations have no effect in some areas on the map. This unpredictability in the approximation made learning difficult for the agent as it lacked spatial awareness. This also highlighted that the agent was not able to close the loop due to approximations.

To resolve this, a closed loop dynamic was implemented. The idea is to follow the same previous intuition, where the last selected node is linked to the newly picked node via the shortest path. After that, the environment automatically links this new node and the starting position via the shortest path between them. This allows us to have a closed loop at each step. When another node is added to the path, the previous segment to the starting position is truncated. The freshly picked node is appended to the path and it is linked via the shortest path to the starting position. For this reason, the last segment evolves dynamically as new nodes are added. Additionally, we tried to reduce the range of distance the agent can take to $100m$ to $300m$. These modifications allowed us to have a clear loop but the same approximation problem occurred making adequate learning not feasible.

2.3 Final Design

2.3.1 Environment Design

At the beginning, for each category, the positions of the nodes are retrieved from the *json* file. After that, it picks randomly a node from the "must have been" places category. These nodes are placed in the same dense area on the map. The intuition is to pick a random node at the beginning to guide the agent to this fixed place. Next, the dead end position nodes are retrieved. These correspond to nodes with only one connected street meaning that the walker would be forced to return backwards. We want to avoid such situations. All this information is computed at the launch of the environment as the graph is fixed and does not change between episodes.

The idea is that the environment proposes to the agent segments of the graph. At each step, a candidate node is proposed to the agent, who can refuse or accept it. If it accepts, using the previously described loop dynamic, the intermediary nodes are added and the accepted node is linked with the previous node and start node via the shortest paths. This maintains a closed loop every time. Next, if the episode is not finished, another segment is proposed. The episode ends when the current loop time exceeds the requested time, or when less than 2 minutes remain before reaching it. We found that being 2 minutes close to the goal is a good compromise as it is difficult to create a path that respects precisely the given delay with segments of variable lengths. A 2 minutes difference threshold allows the agent to terminate naturally, without being over-penalized by small unavoidable timing gaps.

The node proposition strategy is timing-aware and adapts to the current walk total distance. When

less than 50% of the requested time is used, candidate nodes are sampled from a ring between $500m$ and $1000m$ from the last accepted node. Between 50% and 80%, the ring is reduced to between $300m$ and $500m$. Beyond 80% or when the requested time is small, under 15 minutes, the candidate is picked within a $300m$ radius. Nodes present in the current loop are excluded from the proposition to avoid repetitions. However, if there are no other candidates left after this cleaning phase, the system can propose duplicate nodes.

This design strategy allows the agent to learn. Without these techniques, the space of possible proposed nodes would be very large, making the exploration very difficult and the reward signals not informative. It is important to note that this strategy does not provide the optimal solution to the agent. This strategy constraints the candidate nodes to allow more relevant exploration, while still leaving the decision making to the agent. It proposes nodes and does not say anything about the quality of the segments. This step can be seen as an informed sampling rather than a policy as the proposed node may be suboptimal.

Moreover, the agent uses the PPO algorithm from *Stable-Baselines3* and is trained on 40,000 timesteps, corresponding to more than the 50 episodes required. We chose this value as it allows us to observe the convergence on a larger amount of episodes.

2.3.2 State Space

The observation vector contains 16 elements. The first 3 elements encode information about timing such as the required time, the current loop duration and the projected loop duration (with the proposed node). These are all normalized by 70 minutes to maintain the values between 0 and 1. This normalization value was chosen because the agent is trained on requests of 5 to 50 minutes. This range can be exceeded, so using 70 provides a safety margin ensuring nothing exceeds 1.

The next 5 elements are related to the "good walk" criteria. It contains, for each of the 5 quality categories, the proportion of nodes in the shortest path to the candidate node, that belong to that category. The intuition is that the agent observes the path to the proposed node and it judges via these quality criteria, whether the proposition should be accepted or not.

The following 6 elements are 3 pairs of normalized 2 dimension coordinates (x, y) . The first pair corresponds to the last accepted node coordinates, giving the agent intuition about its current position. The second pair is the coordinates of the candidate node, allowing the agent to observe the proposed node position. The third pair corresponds to the coordinates of the guiding "must have been" places. All these coordinates are normalized using min-max scaling with the minimum and maximum coordinates (in latitude and longitude format). These pairs give the agent spatial understanding.

The 15th element is the normalized shortest path distance from the candidate node to the source node. It is divided by 5833, which corresponds to the maximum walkable distance in 70 minutes at $5km/h$. This informs the agent how far the candidate node is from the origin to anticipate if the loop will exceed the required time.

Finally, the last element is a binary value that indicates whether the candidate node is a dead end or not (encoded as 0 or 1). Since dead ends force the walker to retrace their steps, we want to avoid such nodes.

2.4 Action Space

The Action Space corresponds to a binary discrete choice. The agent can either reject (0) or accept (1) the proposed node (and implicitly the segment to reach it).

2.4.1 Reward Function

The objective of the reward function is to make the agent respect the required time, while its path must meet the criteria of a "good walk". When a candidate node is accepted, there are rewards regarding the quality of the path between the last node and the new candidate node. These correspond to the 5 quality ratio observed by the agent. The ratio related to nature and pedestrian zones contributes positively with a weight of 1, rewarding pleasant and safe segments. The must visited ratio is weighted with a factor of 4, as it is important to visit historical places. The amenity ratio is divided by 2 as nodes are abundant and their presence is considered less important. The must avoid places ratio has a weight of 4 and penalizes the agent as it must absolutely avoid such areas. Additionally, a penalty of -1 is given if the node accepts a candidate that sits in a dead end position. When the agent rejects a path, a small penalty of -0.1 is given to avoid that it picks infinitely this action. This ensures that the agent progresses and does not make very long episodes.

At the end of the episode, the timing requirements are inspected. If the final loop duration exceeds the required time, a penalty proportional to the difference is given. If the walk finishes within the two minutes before the targeted time, a proportional positive reward is awarded. Similarly, an additional fixed reward of $+1$ is given if it finishes within 1 minutes. These rewards motivate the agent to respect the required time.

Furthermore, if the option was enabled by the user, the agent receives a reward related to the feedback. The user gives a rating between 1 and 5 and it is respectively mapped to -1 , -0.5 , $+1$, $+2$ or $+4$.

Finally, we also incorporated a reward that is inspired by the potential-based reward shaping. The idea is that it guides the agent toward the "must have been" randomly picked position. The potential function $\phi(s)$ is defined as the Euclidean distance between the last accepted node coordinate and the random "must have been" coordinate. At each step the value $0.99 \times \phi(s) - \phi(s')$ is added to the reward. The intuition is to guide the agent toward the "must have been" area. This strategy of reward shaping is only used for this criterion as other components already provide direct feedback. In contrast, the must have been places are far away from the starting position, in an area positioned at the boundary of the map. As a result, without additional guidance, the agent will never find the "must have been" places as it has low probability of arriving that far from the start by simple exploration. For this reason, this reward shaping allows us to focus on this problem and guide the agent on what the "must have been" places are. Additionally, it might decrease the time taken by the agent to pick such optimal places.

2.4.2 Visualization

We propose terminal and Graphical User Interface modes. The terminal version just outputs the final learning curve. In contrast, the Graphical User Interface version allows visualizing the path during the agent episodes. Moreover, the agent can enable the human rating feedback option in this mode. Every 100 episode, the user can give a rating between 1 and 5. In addition to the learning curves plotted, the reinforcement learning agent performances can be analyzed using *Tensorboard*. We can navigate in the *Tensorboard* interface and observe in detail different metrics such as episode length evolution.

3 Results

The learning curve is produced and displayed in Figure 1. We can observe that the agent converges toward a policy that may generate "good walks", since the reward increases and stabilizes as the number of episodes becomes higher.

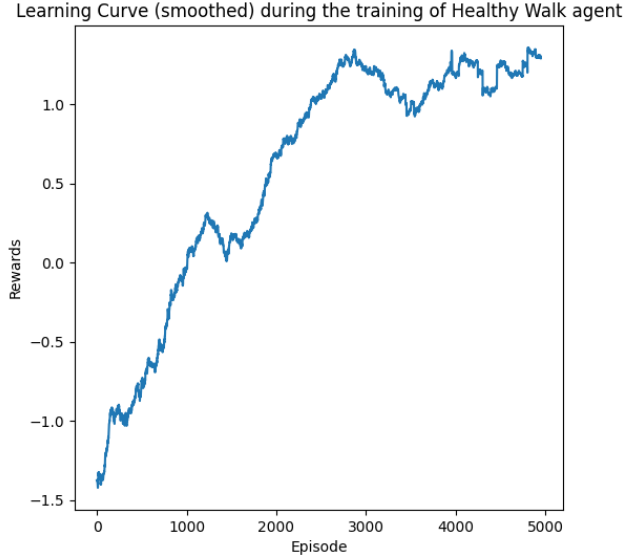


Figure 1: Smoothed learning curve using Moving Average Filter technique with window of size 500. This represents the evolution of rewards obtained by the agent during episodes.

4 Conclusion

In conclusion, the designed environment proposes pleasant healthy walks in Ulqin, Montenegro. For achieving this, the environment suggests a potential portion of paths to an agent that either accepts or rejects them. The agent learns to make these decisions based on a vector of observation containing timing constraints, segment quality and diverse coordinates. The reward function combines segment quality, respect of the timing and reward shaping guiding to "must have been" places. Hence, an agent trained with PPO for 40,000 timesteps progressively converges toward a policy that generates "good walks".

References

Saelens, B. E., & Handy, S. L. (2008). Built environment correlates of walking: A review. *Medicine & Science in Sports & Exercise*, 40(7), S550–S566. <https://doi.org/10.1249/MSS.0b013e31817c67a4>